DAILY JAVA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Java – Beautifully Broken

Quantum-proof handshakes reach JDK 25 next week, and Java starts saying goodbye to Intel Macs

Issue #12 | Fri, 16 Oct 2026 | ⏱️ ~34 min read | Level 3/5

☕ **Today's two stories**

1. **Java's built-in TLS now protects your HTTPS traffic against future quantum computers by default, and next Tuesday it reaches JDK 25.** JDK 27 shipped hybrid post-quantum key exchange (JEP 527) switched on, and we found the backport already sitting in the OpenJDK source for JDK 25.0.5, due 20 October, with the same default. Here's what "hybrid" means, how the handshake changes byte by byte, what we measured, and what to check before your fleet upgrades.
2. **JEP 541 deprecates the JDK's Intel Mac port in JDK 28, and the build now refuses to configure for it unless you insist.** Apple's last Intel-supporting macOS is out, Temurin has already stopped shipping Intel Mac builds of JDK 27, and the last Intel macOS runner on GitHub Actions retires in fall 2027. We read the 14-line change, trace the history of how Java drops a platform, and lay out the options for anyone still on an Intel Mac.
3. **On the radar:** 6 more things that happened, listed at the end.

STORY 1 Post-quantum TLS turns on by default, and JDK 25 gets it next week

JDK 27 & 25 · SECURITY · JEP 527 · OPENJDK 25.0.5 DUE 20 OCT 2026 · ~16 MIN READ

Somewhere, someone may be recording your encrypted traffic today, not to read it now but to read it in fifteen years, once a large enough quantum computer exists. You can't stop the recording. What you *can* do is make sure the keys protecting that traffic were agreed in a way a quantum computer can't undo. Since JDK 27, Java does that by default, without a single line of code from you. On Tuesday, the same change is due to land in JDK 25, the version most teams actually run.

WHAT HAPPENED

JDK 27, released on 15 September 2026, includes **JEP 527: Post-Quantum Hybrid Key Exchange for TLS 1.3**.[1] It adds three new ways for a Java client and server to agree on a secret key during a **TLS** ≈ *Transport Layer Security, the protocol behind HTTPS. It encrypts data between two machines and proves the server's identity* handshake. Each one combines a classic elliptic-curve method with **ML-KEM**, a new algorithm designed to resist quantum computers.[2] The most important detail is the default: one of these, `X25519MLKEM768`, is enabled and preferred out of the box, so "applications that use the `javax.net.ssl` APIs will benefit from these improved algorithms by default".[3]

That's the JDK 27 news, and it's a month old. Here is what's new this week.

On 6 August, Oracle said it wants the same capability in its long-term-support releases and published a timetable: hybrid TLS in JDK 25 with the **October 2026** **Critical Patch Update** ≈ *Oracle's scheduled quarterly security release (CPU), on the third Tuesday of January, April, July and October*, in JDK 21 and 17 in the first half of 2027, and in JDK 11 and 8 in the second half of 2027.[4] The October update is next Tuesday, 20 October.

We checked the open-source side. In the OpenJDK repository for JDK 25 updates, the JEP 527 **backport** ≈ *a change first made in the newest JDK and then copied into an older release line's updates* was committed on 3 August 2026 as "8314323: Implement JEP 527: TLS 1.3 Hybrid Key Exchange", reviewed by four OpenJDK reviewers, while the version file said 25.0.5 with a release date of 2026-10-20.[5] It's included in the latest 25.0.5 build tag, `jdk-25.0.5+7`. And the backported default list is the same as JDK 27's: `X25519MLKEM768` first.[5] In other words, **when your JDK 25 runtime moves to 25.0.5, its HTTPS clients will start offering a post-quantum key exchange by default.**

JDK 21 is on a slower track. Its 25.0.5 equivalent, OpenJDK 21.0.13 (also due on 20 October), carries the backported ML-KEM and ML-DSA algorithms themselves, but not the hybrid TLS groups yet.[6] That matches Oracle's table, which lists ML-KEM and ML-DSA for JDK 21 and 17 in October 2026 and the TLS feature for the first half of 2027.[4]

THE BACKGROUND YOU NEED

Why a quantum computer is a threat *today*

Almost all HTTPS traffic today uses **public-key cryptography** ≈ *cryptography with a key pair: a public key anyone can use and a private key only the owner holds. It lets strangers agree on secrets over an open network* to set up a connection. Two algorithm families do most of that work: RSA, and the elliptic-curve family that includes **ECDHE** ≈ *Elliptic-Curve Diffie-Hellman Ephemeral: each side makes a fresh key pair, they swap public halves, and both compute the same shared secret*. Both rely on maths problems that a sufficiently large quantum computer could solve. JEP 527 names RSA and ECDH as the algorithms quantum computing threatens.[2]

Nobody has such a machine yet. So why change anything now? Because of a threat the JEP calls **harvest now, decrypt later**: an attacker records encrypted traffic today and stores it, waiting for the day the key exchange can be broken.[2] If your traffic carries something that must stay secret for ten or twenty years (health records, financial data, credentials, trade secrets), the clock is already running.

The important word is *key exchange*. In TLS 1.3, the long-term certificate signature proves who the server is, but the actual encryption keys come from a fresh, one-off key exchange. That's the part a recording attacker would need to break, and it's the part JEP 527 changes. Certificates and signatures are a separate migration that this JEP doesn't touch.

Enter ML-KEM, and the KEM idea

In August 2024, the US standards body NIST published FIPS 203 (ML-KEM) and FIPS 204 (ML-DSA), its first post-quantum standards.[4] **ML-KEM** ≈ *Module-Lattice-Based Key-Encapsulation Mechanism, standardised by NIST as FIPS 203. Its security rests on lattice problems that are believed to be hard even for quantum computers. It used to be called Kyber* is the one

that matters for TLS. **ML-DSA** ≈ *Module-Lattice-Based Digital Signature Algorithm (FIPS 204): the post-quantum signature counterpart to ML-KEM* is its sibling for signatures.

ML-KEM is a **KEM** ≈ *Key Encapsulation Mechanism: one side publishes a public key; the other side uses it to produce a random secret plus a "ciphertext" that only the private key holder can turn back into the same secret*, which works slightly differently from Diffie-Hellman. With Diffie-Hellman, both sides contribute a public value and both compute the secret. With a KEM, one side publishes a public key, the other side *encapsulates*, producing a shared secret and a ciphertext, and the first side *decapsulates* that ciphertext with its private key to get the same secret.

Java has had an API for exactly this shape since JDK 21: `javax.crypto.KEM`, added by JEP 452. ML-KEM itself arrived in JDK 24 through JEP 496, and ML-DSA through JEP 497.[4] Here's the API on the JDK 21 runtime we have here, using `DHKEM`, a KEM built from X25519 that JDK 21 already ships. On JDK 24 or later, the same code works with `"ML-KEM"` and an ML-KEM key pair.

```
// Runs on JDK 21+ (no preview flags needed)
KeyPair kp = KeyPairGenerator.getInstance("X25519")
                             .generateKeyPair();
KEM kem = KEM.getInstance("DHKEM");

// Sender: encapsulate against the receiver's public key
KEM.Encapsulated enc =
    kem.newEncapsulator(kp.getPublic()).encapsulate();
byte[] ciphertext = enc.encapsulation();  // sent over the wire

// Receiver: decapsulate with the private key
SecretKey secret =
    kem.newDecapsulator(kp.getPrivate()).decapsulate(ciphertext);
// enc.key() and secret are equal
```

We compiled and ran this on OpenJDK 21.0.12.1: the ciphertext was 32 bytes and both sides ended up with the same secret. Asking that same runtime for `KEM.getInstance("ML-KEM")` throws `NoSuchAlgorithmException`, because ML-KEM isn't in 21.0.12. It's in the 21.0.13 source, due next week.[7,6]

Why "hybrid" and not just ML-KEM?

ML-KEM is young. Elliptic-curve cryptography has had decades of attack and analysis; lattice-based schemes have had far less time in the field. A **hybrid** key exchange runs both and combines the results, so that, as JEP 527 puts it, the connection stays secure as long as *one* of the two algorithms is unbroken.[2] If someone finds a flaw in ML-KEM next year, X25519 still protects you against today's attackers. If a quantum computer arrives, ML-KEM still protects you. The JEP explicitly rules out pure ML-KEM key exchange for now, calling hybrids the option "in greater demand".[2]

The web got here first. Chrome experimented with a hybrid in 2023, enabled it for all desktop clients in 2024, and in Chrome 131 switched to the standardised ML-KEM version, using the code point `0x11EC` for "ML-

KEM768+X25519".[8] That is exactly the group JDK 27 now prefers. Google's engineers also noted a constraint that shaped the design: post-quantum key shares are "too big to be able to offer two post-quantum key share predictions at the same time".[8] Remember that sentence; it's why Java offers one.

From quantum threat to default in your JDK



Sources: Oracle Java blog (6 Aug 2026), Sean Mullan's JDK 27 security post, OpenJDK jdk25u-dev and jdk21u-dev commit history (checked 16 Oct). 20 Oct is the planned release date in the OpenJDK version files.

HOW IT WORKS

Named groups and key shares

A TLS 1.3 connection starts with the client's first message, the **ClientHello** ≈ *the first message of a TLS handshake: the client lists the protocol versions, ciphers and key-exchange methods it supports, and usually includes its key-exchange public values up front*. Two parts of it matter here:

- the **supported groups** list: every key-exchange method the client can do, in preference order. TLS calls these **named groups** ≈ *TLS's name for a key-exchange method, such as x25519 or secp256r1. Each has a registered numeric code point*;
- the **key shares**: the client's actual public values for one or two of those groups, sent immediately so that the handshake can finish in a single round trip.

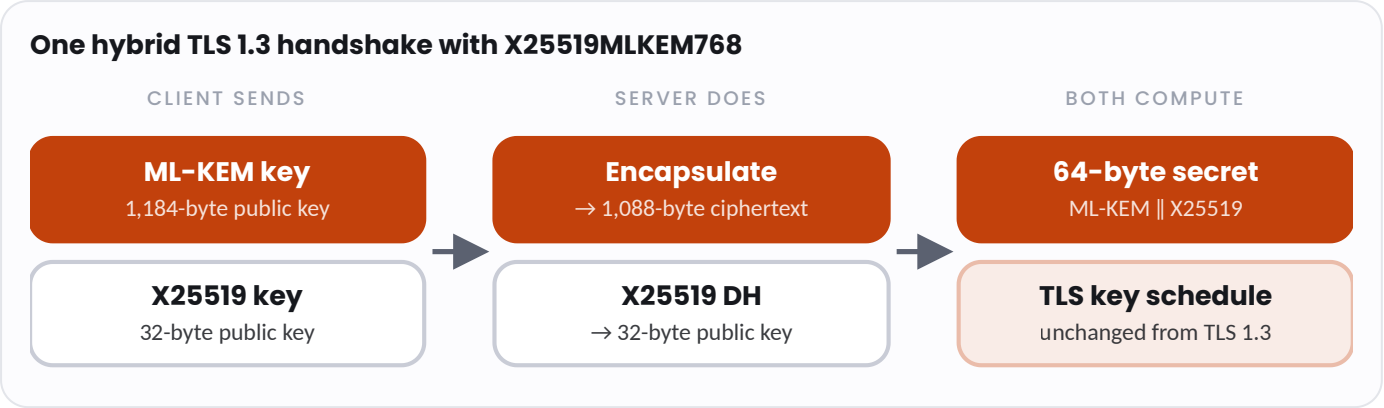
The server picks a group, sends its own value back in the **ServerHello** ≈ *the server's reply in the TLS handshake. In TLS 1.3 it includes the server's key-exchange value for the group it chose*, and both sides feed the shared secret into TLS's key schedule. If the client guessed wrong (the server supports none of the groups it sent shares for), the server answers with a **HelloRetryRequest** ≈ *a TLS 1.3 server reply meaning "please send a key share for this other group instead". It costs an extra network round trip*.

JEP 527 adds three new named groups to the JDK, with names matching the **IANA** ≈ *the Internet Assigned Numbers Authority, which keeps the official registry of TLS code points* registry: `X25519MLKEM768`, `SecP256r1MLKEM768` and `SecP384r1MLKEM1024`.[2] Only the first is in the default list. The default order is `X25519MLKEM768, x25519, secp256r1, secp384r1, secp521r1, x448, ffdhe2048, ffdhe3072, ffdhe4096`, followed in the JEP text by `ffdhe6144, ffdhe8192`.[2] JDK 27 actually removed those last two from the defaults in a separate change, so they'll show up as a difference if you compare JDK 21 and JDK 27 handshake logs.[1] A default JDK 27 client sends **two key shares**: one hybrid `X25519MLKEM768` share and one plain `x25519` share.[3]

Inside the hybrid share

The wire format comes from the IETF document that defines these groups. Sean Mullan's JDK 27 post refers to it as RFC 10024. The latest draft we could open, draft-ietf-tls-ecdhe-mlkem-05 (May 2026), spells out the bytes.[1,9]

For `X25519MLKEM768`, the client's share is an ML-KEM-768 **encapsulation key** ≈ *ML-KEM's name for the public key that the other side encapsulates against* (1,184 bytes) followed by an ordinary X25519 public key (32 bytes): **1,216 bytes**. The server answers with an ML-KEM ciphertext (1,088 bytes) plus its own X25519 public key (32 bytes): 1,120 bytes. Both sides then glue the two 32-byte secrets together, ML-KEM first, into a 64-byte shared secret that goes into the normal TLS 1.3 key schedule.[9]



Two small oddities are worth knowing because they confuse people reading packet captures. First, the X25519 hybrid puts the ML-KEM part *first*, even though the name says X25519 first; the draft calls this ordering "due to historical reasons". Second, the two NIST-curve hybrids put the elliptic-curve part first. The draft explains the difference with certification: whichever algorithm comes first in the combined secret is the one a **FIPS** ≈ *US federal security standards (here FIPS 140) that define how cryptographic modules are validated for government use* -validated implementation must have certified.[9]

Where the JDK's code lives

The backport commit shows the shape of the implementation. It adds new classes inside the JDK's TLS provider, **SunJSSE** ≈ *the JDK's built-in implementation of JSSE, the Java Secure Socket Extension, i.e. javax.net.ssl* : `Hybrid.java` and `HybridProvider.java` (combining two algorithms behind one KEM), `DHasKEM.java` (wrapping elliptic-curve Diffie-Hellman so it looks like a KEM) and `KEMKeyExchange.java`, plus changes to `KeyShareExtension`, `ServerHello` and `NamedGroup`. In all, 22 files and about 1,800 added lines.[5] Wrapping ECDHE as a KEM is the neat trick: once both halves look like KEMs, the TLS code can treat "hybrid" as just another KEM.

What we measured: the ClientHello gets bigger

The hybrid share is big, and that's the main operational change. We captured the ClientHello that a default JDK 21 client sends:

- **422 bytes** on the wire, with key shares for `x25519` (32 bytes) and `secp256r1` (65 bytes).[7]

We have no JDK 27 binary in our sandbox, so we can't capture its ClientHello. But the arithmetic is simple. JDK 27 replaces the 65-byte `secp256r1` share with the 1,216-byte hybrid share (plus 4 bytes of header for each share), adds one entry to the supported-groups list and drops two. That gives roughly **1,570 bytes**, which is about 3.7 times larger and too big for a single standard 1,500-byte Ethernet packet.

Size of a default Java TLS 1.3 ClientHello



JDK 21.0.12.1, connecting to 'localhost' (no SNI extension; real hostnames add a few dozen bytes). The JDK 27 figure is our arithmetic from the IETF draft's 1,216-byte share and the documented default shares, not a capture.

Browsers have carried a share of exactly this size since Chrome 131 switched to ML-KEM, so most of the internet copes.[8] But a ClientHello that spans two packets is new territory for some old **middleboxes** ≈ *network devices in the path between client and server (firewalls, proxies, load balancers, intrusion detectors) that inspect or modify traffic*, and for Java services that talk to old internal endpoints it's the thing to test first.

What happens if the other side doesn't speak hybrid?

Nothing breaks, by design. The hybrid handshake is used only when both peers support the same hybrid group. A JDK 27 server talking to a JDK 17 client simply falls back to classic ECDHE.[10] On the client side, the extra plain `x25519` share means a server that doesn't understand the hybrid can still pick `x25519` without another round trip.

There's one subtle side effect. JDK 21 also sent a `secp256r1` share; JDK 27 doesn't. A server that *only* accepts `secp256r1` will now need a HelloRetryRequest. We reproduced the mechanism on JDK 21 with a server restricted to `secp384r1` and a client sending `x25519` and `secp256r1` shares: the debug log shows `HelloRetryRequest selected named group: secp384r1`, then a second ClientHello.[7] The handshake succeeds; it just costs one extra round trip.

WHAT IT MEANS FOR YOU

If you're on JDK 27, you already have it. Any code that uses the JDK's TLS stack through `javax.net.ssl` (for example `java.net.http.HttpClient` or `HttpsURLConnection`) offers `X25519MLKEM768` first, without code changes, unless you have overridden the named groups.[11] Whether you actually get a post-quantum connection depends on the server supporting the same group.

If you're on JDK 25, plan for it next week. Your first update after 20 October should bring the same default, if your build is made from the OpenJDK 25.0.5 source we checked.[5] Oracle says its JDK 25 will reach parity with JDK 27 in the October CPU.[4] Other distributions build from the same jdk25u source, but read your vendor's 25.0.5 release notes to confirm. Treat the upgrade like any TLS change: run it through staging, with special attention to calls to old internal services and anything behind TLS-inspecting firewalls.

If you're on JDK 21 or 17, there's nothing to do yet. 21.0.13 brings the ML-KEM and ML-DSA algorithms, but no hybrid TLS by default.[6] Oracle's plan is the first half of 2027 for JDK 21 and 17, and the second half of 2027 for JDK 11 and 8.[4]

To check what you actually negotiated, turn on handshake logging with `-Djavax.net.debug=ssl:handshake`. The ClientHello's `key_share` section lists a `"named group"` for each share, and the ServerHello's `key_share` shows the one the server chose. That's how we read the shares in our JDK 21 tests.[7]

To control it, there are two documented switches. The `jdk.tls.namedGroups` system property replaces the default list for the whole JVM, and `SSLParameters.setNamedGroups(...)` sets it per socket or engine.[2,3]

Choosing key-exchange groups

Opt out JVM-wide (classic only)

```
java -Djdk.tls.namedGroups=\
x25519,secp256r1,secp384r1 \
  -jar app.jar

# Diagnose a handshake:
java -Djavax.net.debug=\
ssl:handshake -jar app.jar
```

Per socket (JDK 27+, 25.0.5+)

```
SSLSocket s = (SSLSocket) SSLContext
    .getDefault().getSocketFactory()
    .createSocket(host, 443);
SSLParameters p = s.getSSLParameters();
p.setNamedGroups(new String[] {
    "SecP256r1MLKEM768",
    "X25519MLKEM768",
    "secp256r1", "x25519" });
s.setSSLParameters(p);
```

The right-hand list is the example from the Inside.java announcement. Leaving a hybrid name out of the list is how you turn it off.

Performance. Oracle's team has been adding **intrinsics** *≈ hand-written machine code that the HotSpot JIT swaps in for specific Java methods, here for the hot loops of ML-KEM and ML-DSA* for these algorithms. A 30 September Inside.java post shows the gains on x64 and AArch64 in charts (we couldn't read exact figures from the text).[12] Sean Mullan lists JDK 27 performance work on SHA-3, ML-KEM, ML-DSA, X25519 and Ed25519.[1] In the JDK 25 and 21 update repositories, several of these optimisations (AVX-512 and AArch64 intrinsics for ML-KEM, an optimised Java ML-KEM) were backported alongside the algorithms.[5,6] The cost that remains is mostly bytes on the wire, not CPU.

💥 Watch out for

- **This protects confidentiality, not identity.** Certificates are still RSA or ECDSA. Post-quantum signatures in TLS are a separate, later step.
- **It only covers the JDK's own TLS.** Libraries that bring their own TLS engine (Netty with OpenSSL via netty-tcnative, the Bouncy Castle JSSE provider, a sidecar or load balancer that terminates TLS for you) follow their own settings. If your service sits behind a proxy, the internet-facing connection is protected only if the *proxy* speaks hybrid.
- **Pinned group lists hide the feature.** If you or a framework already sets `jdk.tls.namedGroups`, your list wins and the hybrid isn't offered.[11] Search your launch scripts and Helm charts for that property.

WHAT HAPPENS NEXT

The immediate milestone is **Tuesday 20 October**: OpenJDK 25.0.5 and 21.0.13, and Oracle's October CPU.[5,6,4] After that, watch the jdk21u and jdk17u repositories for a JEP 527 backport (Oracle targets the first half of 2027), and the IETF for pure-ML-KEM groups, which JEP 527 left open as possible future work.[2] In JDK 28, Oracle engineers continue the intrinsics work.[12] And keep Oracle's own caveat in mind: the platform "can enable post-quantum cryptography, but it does not force its use".[4]

📖 Words from this story

TLS: Transport Layer Security, the protocol behind HTTPS.

Public-key cryptography: cryptography with a public and a private key, used to agree on secrets over an open network.

ECDHE: Elliptic-Curve Diffie-Hellman Ephemeral; both sides exchange fresh public values and compute the same secret.

ML-KEM: NIST's post-quantum key-encapsulation standard (FIPS 203), formerly Kyber.

ML-DSA: NIST's post-quantum signature standard (FIPS 204).

KEM: Key Encapsulation Mechanism; encapsulate against a public key to get a secret plus a ciphertext, decapsulate with the private key.

ClientHello: the client's first TLS message, listing what it supports and carrying key shares.

Named groups: TLS's term for key-exchange methods such as x25519 or X25519MLKEM768.

ServerHello: the server's reply, with its key share for the chosen group.

HelloRetryRequest: a server request for a key share in a different group; costs a round trip.

Encapsulation key: ML-KEM's public key.

FIPS: US federal standards for validating cryptographic modules.

SunJSSE: the JDK's built-in implementation of `javax.net.ssl`.

Middleboxes: firewalls, proxies and other devices that inspect traffic in transit.

Intrinsics: hand-written machine code the JIT substitutes for specific Java methods.

Critical Patch Update (CPU): Oracle's quarterly scheduled security release.

Backport: a change copied from the newest JDK into an older release line's updates.

IANA: the registry keeper for TLS code points.

STORY 2

JEP 541: Java begins saying goodbye to the Intel Mac

JDK 28 · PLATFORM SUPPORT · JEP 541 (TARGETED) · IMPLEMENTED 27 AUG 2026 · ~13 MIN READ

If you bought one of the last Intel MacBooks, it has had a good run, and it still runs the latest Java. But the ground under it is moving. Apple's newest macOS no longer installs on Intel Macs, Eclipse Temurin didn't build JDK 27 for them, and in JDK 28 the OpenJDK build itself will stop and ask, in effect, "are you sure?" when you try to compile Java for one. Here's what JEP 541 does, what it doesn't do, and how long your Intel Mac can keep running Java.

WHAT HAPPENED

JEP 541: Deprecate the macOS/x64 Port for Removal is targeted to JDK 28, which is due in March 2027.[13,14] It started life on 30 June as JEP draft 8386091, which stated plainly that "Oracle engineers will stop maintaining the macOS/x64 port as of JDK 27".[16] It became a Candidate in late July[15] and was reported as Targeted to JDK 28 in InfoQ's roundup for the week of 17 August; Inside.java listed it as "JEP targeted to JDK 28" on 29 September.[14,13]

The JEP's reasoning, as quoted by OSnews, is short: "Apple has transitioned its hardware products to the AArch64 processor architecture," and "Maintaining the port is a significant burden." The port isn't removed yet; the stated intent is to remove it in a future release.[17,15]

The implementation is already in the JDK 28 mainline. Oracle's Mikael Vidstedt committed it on 27 August 2026 as "8386092: Implement JEP 541: Deprecate the macOS/x64 Port for Removal". It touches three files and adds 15 lines.[18] We'll read all of them below.

THE BACKGROUND YOU NEED

What a "port" is

The JDK isn't one program that runs everywhere. Java *bytecode* is portable; the JVM that runs it is not. Each **port** ≈ *one combination of operating system and CPU architecture that the JDK is built, tested and shipped for, such as linux-x64 or macos-aarch64* needs its own native code: the JIT compilers must emit machine instructions for that CPU, and the runtime must call that operating system's APIs for threads, memory and signals. OpenJDK groups this code by OS and by CPU, so "macOS/x64" means "the macOS-specific code plus the Intel-64 CPU code, built and tested together".

Every port costs something: build machines, test machines, engineers who notice when something breaks. That's why the JDK has a formal process for dropping one.

How Java drops a platform

The process has two steps, each a JEP. First, a "deprecate for removal" JEP makes the build fail by default for that platform, so anyone still building it has to opt in knowingly. Then, a release or more later, a second JEP deletes the code.

It has happened twice before in recent memory:

- **Solaris and SPARC**: JEP 362 deprecated the Solaris/SPARC, Solaris/x64 and Linux/SPARC ports in JDK 14, with the build error "The Solaris and SPARC ports are deprecated and may be removed in a future release", suppressible with `--enable-deprecated-ports=yes`. JEP 381 then removed them.[20]
- **32-bit Windows**: JEP 449 deprecated the Windows 32-bit x86 port in JDK 21 with the same mechanism.[21] JEP 479 removed its code and build support in JDK 24.[22]

JEP 541 follows the Windows precedent explicitly; InfoQ describes it as modelled on JEP 449.[15,14]

🪟 Windows x86-32

- Deprecated: JEP 449, JDK 21
- Configure error unless opted in
- Removed: JEP 479, JDK 24

```
--enable-deprecated-ports=yes
```

VS

🍎 macOS/x64

- Deprecated: JEP 541, JDK 28
- Same configure error, Zero exempt
- Removal: 'a future release'

```
--enable-deprecated-ports
```

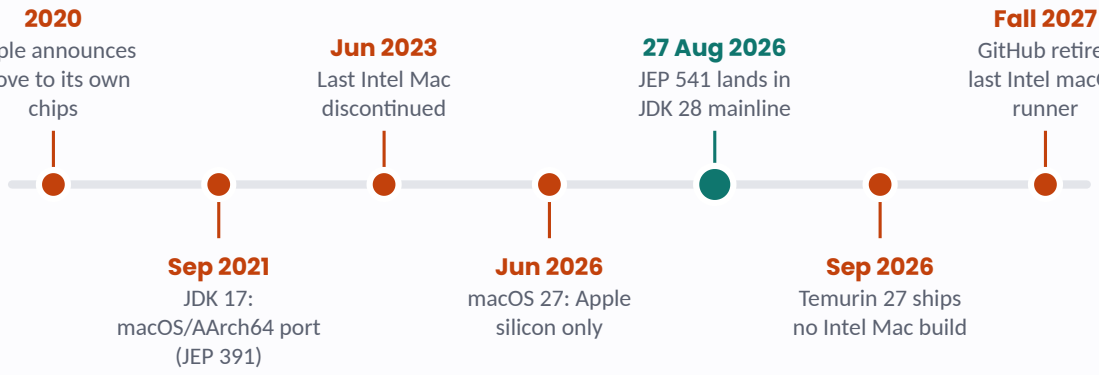
Apple's side of the story

The JDK followed Apple's lead. Apple announced its move from Intel to its own ARM-based chips in 2020.[24] OpenJDK added a native AArch64 ≈ *the 64-bit ARM architecture used by Apple silicon (M1 and later), also called arm64* port for macOS in JDK 17 (JEP 391).[23] Apple sold its last Intel Mac, the 2019 Mac Pro, until June 2023.[16]

In June 2026, Apple confirmed that macOS 27 "Golden Gate" is the first macOS that runs only on Apple silicon, and the last with full Rosetta 2 ≈ *Apple's translation layer that lets apps compiled for Intel run on Apple-silicon Macs* support. Apple said at WWDC 2025 that Rosetta would be available "through macOS 27", after which only a subset aimed at old games remains.[24]

That second point matters to Java in a sneaky way. An Intel JDK can run on an Apple-silicon Mac under Rosetta, which is one way to keep old x64-only native libraries working. That route is closing too.

The Intel Mac and the JDK



Sources: MacRumors (10 Jun 2026), JEP 391, The Register (30 Jun 2026), openjdk/jdk commit ee2ebf6, Adoptium (2 Sep 2026), GitHub changelog (19 Sep 2025). JDK 28 itself is due in March 2027.

HOW IT WORKS

The whole change, read line by line

The commit is small enough to read in full.[18] The heart of it is in `make/autoconf/platform.m4`, part of the JDK's **autoconf** *≈ the GNU tool that generates the JDK's `configure` script, which checks your machine and settings before a build* setup. Before the change, a function called `PLATFORM_CHECK_DEPRECATION` contained only a comment: "There are no deprecated ports. Implement the deprecation warnings here." (We checked: that's still exactly what JDK 27's GA source says.[19]) Now it reads, in essence:

```
if target OS is macosx and target CPU is x86_64:
  if the JVM variant requested is not "zero":
    if --enable-deprecated-ports=yes:
      WARN  "The macOS/x64 port is deprecated and may be
             removed in a future release."
    else:
      ERROR "...  Use --enable-deprecated-ports to
             suppress this error."
```

So if you run `bash configure` for an Intel Mac target on the JDK 28 source tree, configuration stops with an error. Add `--enable-deprecated-ports=yes` and it continues, printing the same message as a warning.



There's one twist that the earlier deprecations didn't have: the **Zero exemption**. **Zero** *≈ an interpreter-only HotSpot variant written in C++ with essentially no assembly code. It runs anywhere a C++ compiler does, but much more slowly because it has no JIT* is HotSpot's "no assembler" variant. Its author described it in 2009 as "an interpreter-only port of HotSpot that uses no assembler".[25] A Zero build doesn't use the x64-specific JIT and interpreter code at all, so it doesn't carry the maintenance burden the JEP is about. The code comment says variants "have not been parsed yet" at that point, so it checks the raw `--with-jvm-variants` option and lets only a plain `zero` through.[18] In practice, that's a curiosity: Zero is far slower than a normal JVM, and a 2024 OpenJDK discussion described its performance as very poor compared with the default.[26]

The other two files are about Oracle's and OpenJDK's own machinery:

- `.github/workflows/main.yml` adds `macos-x64` to `EXCLUDED_PLATFORMS`, next to `alpine-linux-x64`. OpenJDK's GitHub Actions pre-submit testing no longer builds and tests Intel macOS by default for pull

requests.[18]

- `make/conf/jib-profiles.js`, which describes the build profiles used by Oracle's internal build system, adds `--enable-deprecated-ports` to the macOS profile.[18] So, at least for now, Oracle's own macOS build configuration still opts in and configures cleanly. The JEP's warning in the earlier draft still applies, though: The Register quotes it as giving "no guarantee that the port will build, much less function".[16]

What "deprecated" doesn't mean

It doesn't change a running JVM. Nothing in the commit touches HotSpot or the class library, so a macOS/x64 JDK that someone builds and ships still works the same way. There's no runtime warning and no new flag for applications. The deprecation acts on **people who build the JDK**, and the real-world effect comes through the vendors who decide what to build.

WHAT IT MEANS FOR YOU

The effect is already visible in the distributions, and it's not uniform.

Eclipse Temurin went a release early. On 2 September, Adoptium's George Adams announced that Temurin JDK 27 "will not be built or published for macOS x64 (Intel)", giving two reasons: Intel Mac hardware is "becoming increasingly difficult to source and maintain", and the upstream port is being dropped. Earlier Temurin releases "are unaffected by this announcement", and Intel users of JDK 27 are pointed to the aarch64 build.[27]

Amazon Corretto 27, on the other hand, still lists a macOS x64 download (both `.pkg` and `.tar.gz`) alongside macOS aarch64.[28]

Oracle stopped *maintaining* the port as of JDK 27, according to the JEP draft.[16] We couldn't confirm from our sandbox whether Oracle's own JDK 27 download page offers an Intel Mac build. Reports say JDK 25 LTS updates are expected to continue on Intel Macs.[16]

CI is the next pressure point. If your team tests on Intel macOS in GitHub Actions, note that the `macos-13` image was scheduled for retirement by 4 December 2025 and `macos-15-intel` is the remaining Intel option. GitHub says it "will no longer support this architecture on macOS after the macOS 15 runner image is retired in Fall 2027".[29]

Java isn't alone in this. The Register lists the same move in other ecosystems: Rust demoted Intel macOS to **Tier 2** ≈ *a lower support level: still built (and in some projects still release-blocking), but with weaker testing guarantees than Tier 1* with Rust 1.90, Python has listed it as Tier 2 since November 2025, and Node.js moved it to Tier 2 in May 2026, planning to treat Intel Macs as experimental from early 2028.[16]

If you ship native code, look at your **JNI** ≈ *Java Native Interface: the classic way for Java code to call C/C++ libraries* or **FFM** ≈ *the Foreign Function & Memory API, the modern replacement for JNI* libraries. Anything built only for `x86_64` macOS (an old `.dylib`, macOS's shared-library format) runs on an Apple-silicon Mac only through an Intel JDK under Rosetta, and that path is closing on both ends: fewer Intel JDKs, and Rosetta's full support ends with macOS 27.[24] Rebuild as `arm64` or as a **universal binary** ≈ *a single macOS file containing code for both Intel and Apple-silicon CPUs* now, while you still have both kinds of machine to test on.

🌍 Your options if you develop on an Intel Mac

1. **Stay on an LTS JDK.** JDK 25 (and 21) builds for macOS/x64 still exist from several vendors, and existing releases aren't being withdrawn. That's enough runway for most laptops.[27,16]
2. **Pick a distribution that still ships it.** Corretto 27 has an x64 macOS download today.[28] Check before each upgrade, though; vendors can change their plans.
3. **Run newer JDKs in Linux.** A Docker container or VM running linux-x64 is a fully supported port and is what most servers use anyway. The Register suggests exactly this.[16]
4. **Build it yourself**, with `--enable-deprecated-ports=yes`, until the port is removed. Accept that nobody upstream is testing it.[18]
5. **Plan the hardware.** If newer JDKs matter to you, an Apple-silicon Mac runs the fully supported macOS/AArch64 port.

💥 Watch out for

- **Deprecated is not removed.** JDK 28 can still be built for Intel Macs; it just needs the opt-in flag. Removal needs a separate JEP in a later release.[15]
- **The aarch64 JDK won't run on an Intel Mac.** Temurin's advice to "use the macOS aarch64 JDK 27 port instead" only helps if your Mac has Apple silicon.[27]
- **Don't confuse this with the 32-bit x86 removal.** Linux and Windows on Intel/AMD 64-bit (x64) are not affected at all. Only the macOS + x64 combination is being deprecated.
- **Rosetta isn't a long-term plan.** Running an x64 JDK on Apple silicon via Rosetta works on macOS 27, but Apple only promised full Rosetta support through macOS 27.[24]

WHAT HAPPENS NEXT

JDK 28 is due in March 2027 with the port deprecated. A follow-up JEP to actually remove the code would be the next step; on the Windows precedent, that took three releases.[21,22] Meanwhile, watch the other vendors (Azul, BellSoft, Microsoft, SAP) for their JDK 28 platform lists, and Adoptium's tracking issue for whether it extends the Intel Mac cut to its LTS lines.[27]

📖 Words from this story

Port: one OS + CPU combination the JDK is built and tested for, such as macos-x64.

AArch64: 64-bit ARM, the architecture of Apple silicon (also called arm64).

Rosetta 2: Apple's translation layer for running Intel apps on Apple silicon.

autoconf: the GNU tool that generates the JDK's `configure` script.

Zero: an interpreter-only HotSpot variant written in C++, with no JIT and almost no assembly.

JNI: Java Native Interface, the classic way to call C/C++ from Java.

FFM: the Foreign Function & Memory API, JNI's modern replacement.

Universal binary: one macOS file with code for both Intel and Apple silicon.

Tier 2: a lower platform-support level with weaker testing guarantees.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **JDK 28 early-access build 20 was tagged on 15 October:** it's the first build that includes JEP 544's ahead-of-time code compilation (our issues #2 and #10).[30]
- **JEPs 543, 545 and 546 have finished their Proposed-to-Target reviews (13–14 October):** structured concurrency (final) and the two ZGC startup and heap-sizing JEPs await the "Targeted" announcement, which hadn't appeared on Inside.java by 16 October.[31,13]
- **Shenandoah loses an experimental flag in JDK 28:** `ShenandoahImplicitGCInvokesConcurrent`, which controlled whether internally triggered GCs run concurrently or stop the world, was removed from the JDK 28 mainline on 12 October.[32]
- **GlassFish 8.0.5:** bug fixes, dependency upgrades, ZIP64 header updates and faster deployment for large and small applications.[31]
- **TornadoVM 7.2.0 and 7.1.0:** the GPU-acceleration toolkit for Java adds HalfFloat matrix-multiply load methods and more accurate CUDA dispatch timing.[31]
- **Grails 8.0.0 GA (8 October):** requires JDK 21 and Gradle 9.7 and treats all controllers as REST resources.[31]

---

1. **JDK 27 Security Enhancements**, Sean Mullan, 15 Sep 2026.
2. **JEP 527: Post-Quantum Hybrid Key Exchange for TLS 1.3**, OpenJDK. The version we could open was last updated 8 Dec 2025.
3. **Post-Quantum Hybrid Key Exchange for TLS 1.3**, Inside.java, 17 Feb 2026.
4. **Post-Quantum Cryptography in Long-Term Support JDK Releases**, Aurelio Garcia-Ribeyro, Oracle Java blog, 6 Aug 2026.
5. **openjdk/jdk25u-dev commit 83e90e7 "8314323: Implement JEP 527: TLS 1.3 Hybrid Key Exchange"**, 3 Aug 2026 (backport of mainline 21dc41f). We checked it with git: present in tag jdk-25.0.5+7; `make/conf/version-numbers.conf` at that commit says update 5, date 2026-10-20; `NamedGroup.java` default list starts with X25519MLKEM768. ML-KEM intrinsics backports seen in the same history (8360934, 8374755, 8383635).
6. **openjdk/jdk21u-dev commits 6de158a "Implement JEP 496"** (30 Jul 2026) and **ae88cd9 "Implement JEP 497"** (31 Jul 2026). Checked with git: both in tag jdk-21.0.13+8 (version file: 21.0.13, 2026-10-20); no JEP 527 backport in jdk21u-dev as of 16 Oct. ML-KEM AVX-512/aarch64 intrinsics and optimisations also backported (Aug 2026).
7. Our own test on OpenJDK 21.0.12.1 (Ubuntu build), 16 Oct 2026: KEM API demo, a raw-socket capture of the default ClientHello, `javax.net.debug=ssl:handshake` logs, and a HelloRetryRequest reproduction with mismatched groups.

8. **A new path for Kyber on the web**, Google Online Security Blog (Chrome team), 13 Sep 2024.
9. **draft-ietf-tls-ecdhe-mlkem-05: Post-quantum hybrid ECDHE-MLKEM Key Agreement for TLSv1.3**, IETF, 26 May 2026 (sections 4.1–4.3, 5, 7).
10. **Java 27 Turns On Post-Quantum TLS by Default**, Netics Labs, Sep 2026.
11. **Quality Outreach Heads-up - JDK 27: Post-Quantum Hybrid Key Exchange for TLS 1.3**, Ana-Maria Mihalceanu, Inside.java, 17 May 2026.
12. **Faster Post-Quantum Cryptography with JDK Intrinsics**, Mark Powers, Inside.java, 30 Sep 2026.
13. **Inside.java front page**, checked 16 Oct 2026 ("JEP targeted to JDK 28: 541", 29 Sep 2026; no targeted announcements for JEPs 543/545/546 yet).
14. **Java News Roundup: JDK 27-RC1, OpenJDK JEPs, Jakarta EE, BellSoft, Helidon, Micrometer, Tika 4.0**, InfoQ, 25 Aug 2026.
15. **Java News Roundup (week of 20 July 2026)**, InfoQ, 27 Jul 2026.
16. **No more Java refills for Intel Macs after JDK 27, says Oracle**, Tim Anderson, The Register, 30 Jun 2026.
17. **Java deprecates support for macOS x86**, Thom Holwerda, OSnews, 27 Jul 2026.
18. **openjdk/jdk commit ee2ebf6 "8386092: Implement JEP 541: Deprecate the macOS/x64 Port for Removal"**, Mikael Vidstedt, 27 Aug 2026 (diff read with git).
19. `make/autoconf/platform.m4` at tag jdk-27-ga in openjdk/jdk, read with git on 16 Oct 2026.
20. **JEP 362: Deprecate the Solaris and SPARC Ports**, OpenJDK.
21. **JEP 449: Deprecate the Windows 32-bit x86 Port for Removal**, OpenJDK (delivered in JDK 21).
22. **JEP 479: Remove the Windows 32-bit x86 Port**, OpenJDK (delivered in JDK 24).
23. **JEP 391: macOS/AArch64 Port**, OpenJDK (delivered in JDK 17).
24. **macOS 27 Golden Gate Is the Last to Support Intel Apps via Rosetta 2**, Hartley Charlton, MacRumors, 10 Jun 2026.
25. **Review request: Zero assembler port**, Gary Benson, hotspot-dev mailing list, 15 Jul 2009.
26. **New candidate JEP: 501: Deprecate the 32-bit x86 Port for Removal (discussion)**, jdk-dev mailing list, 15 Nov 2024.
27. **JDK 27 Will No Longer Be Built for macOS x64**, George Adams, Eclipse Adoptium, 2 Sep 2026.
28. **Amazon Corretto 27 downloads**, AWS documentation, checked 16 Oct 2026.
29. **GitHub Actions: macOS 13 runner image is closing down**, GitHub changelog, 19 Sep 2025.
30. **openjdk/jdk tag jdk-28+20** (commit 84054db, 15 Oct 2026), checked with git: contains **f6fa653 "Implement JEP 544"**.
31. **Java News Roundup: Grails 8, Open Liberty, GlassFish, Infinispan, LangChain4j, Spock**, InfoQ, 12 Oct 2026.
32. **openjdk/jdk commit 26582da "8393554: Shenandoah: Remove ShenandoahImplicitGCInvokesConcurrent option"**, William Kemper, 12 Oct 2026.